

Uart_Esp32 - Projektdokumentation - Firmware

Firmwarestand v0.9 - UART-Routingkorrektur und Live-Status

Projektname: Uart_Esp32

Dokumentenart: Projektdokumentation

Thema: Firmware fuer ESP32-WROOM-32 / NodeMCU-32S - UART-Routingkorrektur, getrennte Einstellungen und Decoder-Dauerbetrieb

Erstellungsdatum: 2026-09-17

Autor / verantwortliche Person: OpenAI ChatGPT (technische Erstellung)

Version: v0.9

Status: Entwurf / lokale Build- und Hardwarepruefung ausstehend

Aenderungsdatum: 2026-09-17

Ablageort: Projektordner Uart_ESP32/docs

Referenzen: Regeln_Projektdokumentation_PDF_DOCX_Pflicht(5).pdf; ESP32_UART_Protokoll_Entschluesselung(1).pdf; Projektstand v0.8

Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.1	2026-09-17	OpenAI ChatGPT	Entwurf	Reduzierter WROOM32-Projektstand erstellt
v0.2	2026-09-17	OpenAI ChatGPT	Entwurf	Projekt in Uart_Esp32 umbenannt und Dokumentationsregeln umgesetzt
v0.3	2026-09-17	OpenAI ChatGPT	Entwurf	Buildfehler durch typensichere Integer-Vergleiche behoben
v0.4	2026-09-17	OpenAI ChatGPT	Entwurf	Arduino-Secrets, NVS-Fallbacks und WLAN-Provisionierung ergaenzt
v0.5	2026-09-17	OpenAI ChatGPT	Entwurf	WLAN-State-/Fallback-Logik, Recovery-AP, DHCP-Fallback und Resetdiagnose ergaenzt
v0.6	2026-09-17	OpenAI ChatGPT	Entwurf	Compilerfehler in markWifiConnected() behoben
v0.7	2026-09-17	OpenAI ChatGPT	Entwurf	UART Raw/Sniffer, Protokoll-Decoder, Feld-Aliase, Kalibrierung und UART-Webseite ergaenzt
v0.8	2026-09-17	OpenAI ChatGPT	Entwurf	UART-Konfiguration und Live-Monitor getrennt; Raw/Decoder als persistenter Dauerbetrieb dokumentiert
v0.9	2026-09-17	OpenAI ChatGPT	Entwurf	UART-Unterseiten eindeutig geroutet; Status-API repariert; Livezaehler und API-Diagnose verbessert

Ziel des Standes v0.9

Der Projektstand v0.9 korrigiert einen Integrationsfehler der UART-Weboberflaeche. Bei der verwendeten ESPAsyncWebServer-Version konnte die zuerst registrierte kurze Route /uart auch Anfragen an die Unterseiten /uart/settings und /uart/status abfangen. Das erklart das beobachtete Verhalten: Im Browser stand zwar /uart/settings, angezeigt wurde aber erneut die Monitorseite. Gleichzeitig erhielt das JavaScript fuer /uart/status HTML statt JSON; dadurch blieben Byte- und Paketaehler sichtbar auf 0, obwohl im Raw-Puffer bereits UART-Daten ankamen.

Ab v0.9 werden die spezifischen Routen zuerst registriert. Die Live-Seite besitzt nun die eindeutige Route /uart/monitor; /uart ist nur noch ein kompatibler Redirect. Die Status-API wird vor allen anderen UART-Routen registriert und liefert explizit JSON mit Cache-Control: no-store. Die Monitorseite prueft den Content-Type und zeigt den Zustand der Status-API sichtbar an.

Der Decoder selbst bleibt ein echter Dauerbetrieb: Nach Aktivierung wird er in der ESP32-Hauptschleife kontinuierlich ausgefuehrt, unabhaengig davon, ob eine Browserseite geoeffnet ist. Der gewaehlte Modus bleibt in NVS gespeichert und wird nach einem Neustart automatisch wieder gestartet.

UART-Betriebsarten

Modus	Verhalten
Aus	Zusaetzliche Hardware-UART ist deaktiviert.

Raw / Sniffer	Bytes werden empfangen und in einem 512-Byte-Ringpuffer gespeichert. Die letzten 256 Bytes werden live als HEX und ASCII angezeigt.
Protokoll-Decoder	Raw-Erfassung bleibt aktiv; zusätzlich werden Pakete synchronisiert, geprüft und die bestätigten Hauptpakete dekodiert.

UART0 bleibt fuer den seriellen Debug-Monitor reserviert. Fuer die Zusatzfunktion sind Hardware-UART1 und UART2 wählbar. RX und TX sind frei konfigurierbar; TX kann mit -1 deaktiviert werden. Standard ist RX GPIO16, TX deaktiviert, 115200 Baud, 8N1.

Dauerbetrieb: Die UART-Verarbeitung wird in loop() kontinuierlich aufgerufen und besitzt keine Messanzahl oder Laufzeitbegrenzung. Der 512-Byte-Ringpuffer kann dabei nicht volllaufen: Sobald er gefüllt ist, werden nur die jeweils ältesten Rohbytes ueberschrieben. Die Dekodierung selbst laeuft weiter. Das Oeffnen oder Schliessen der Weboberflaeche beeinflusst den Decoder nicht.

Nicht zugelassen sind GPIO6..11 wegen des externen Flashs sowie GPIO1/3, damit der Debug-UART nicht kollidiert. GPIO34..39 sind nur als RX geeignet, da sie eingangs-only sind.

Decoder gemaess Messunterlage

Der Decoder verwendet die in ESP32_UART_Protokoll_Entschluesselung(1).pdf dokumentierte Struktur:

1. Suche nach FF.
2. Folgendes Byte muss FB sein.
3. Typ und Laengenbyte lesen.
4. Gesamtlänge als 4 + Länge bestimmen.
5. Vollständiges Paket sammeln.
6. XOR-Pruefsumme ueber Byte 6 bis zum Byte vor der Pruefsumme bilden.
7. Nur gueltige Pakete weiter auswerten.

Fuer das bestaetigte Hauptpaket Typ 0x10, Länge 0x15 ergibt sich eine Gesamtlänge von 25 Bytes. Die sechs 16-Bit-Felder liegen Little Endian ab Byte 12:

Verifikationshinweis: Die Implementierung folgt der im PDF textlich angegebenen XOR-Regel (Byte 6 bis zum Byte vor der Pruefsumme). Die im PDF abgedruckte einzelne Beispiel-Bytefolge ergibt bei einer unabhaengigen Nachrechnung nicht denselben Endwert wie das dort gezeigte letzte Byte. Deshalb zeigt die Firmware gueltige/ungueltige Paketaeähler separat an; der Hardwaretest muss bestaetigen, ob die XOR-Grenzen fuer die realen Mitschnitte exakt stimmen. Der Raw-Modus bleibt hiervon unberuehrt.

Technisches Feld	Byteposition	Behandlung
Feld 1	12 / 13	uint16 LE, normiert
Feld 2	14 / 15	uint16 LE, normiert
Feld 3	16 / 17	uint16 LE, normiert
Feld 4	18 / 19	uint16 LE, normiert
Feld 5	20 / 21	uint16 LE, normiert
Feld 6	22 / 23	uint16 LE, nur Rohwert
Checksumme	24	XOR

Normierung und Kalibrierung

Feld 1 bis Feld 5 werden stueckweise linear auf -1.0 ... +1.0 normiert. Minimum, Mitte und Maximum sind fuer jedes Feld separat einstellbar und werden in NVS gespeichert. Als Startwerte werden die Messwerte aus der Referenz verwendet:

Feld	Minimum	Mitte	Maximum
Feld 1	1000	1503	2000
Feld 2	1000	1486	2000
Feld 3	1000	1493	2000
Feld 4	1006	1511	2000

Feld 5	1000	1498	2000
--------	------	------	------

Eine einstellbare Totzone wird nach der Normierung angewendet; Standard ist 0.03. Feld 6 wird bewusst nicht normiert, da die Messunterlage seine Bedeutung noch nicht abschliessend klaert.

Feld-Aliase

Fuer jedes Feld steht ein Textfeld mit maximal 32 Zeichen bereit. Die Darstellung bleibt immer zweistufig:

- feste technische Kennung: Feld 1, Feld 2, ... Feld 6
- optionaler Benutzer-Alias: z. B. Roll, Pitch, Schieber, noch unbekannt

Der Alias ersetzt die technische Kennung nicht. Aliase werden in NVS gespeichert und im Webinterface sowie ueber MQTT ausgegeben.

Webinterface

Ab v0.9 sind Konfiguration und Beobachtung bewusst getrennt. Dadurch bleibt die Live-Seite uebersichtlich, waehrend die umfangreichen Einstellungen auf einer eigenen Seite liegen.

UART Monitor `/uart/monitor``

Die Monitorseite enthaelt nur Laufzeitinformationen:

- aktuellen UART-/Decoder-Status,
- fortlaufenden Bytezaehler,
- Feld 1 bis Feld 6 mit technischer Kennung, Alias, Rohwert und - fuer Feld 1 bis 5 - normiertem Wert,
- Paketaehler fuer gesamt, gueltig, ungueltig und 0x10/0x15,
- die letzten 256 Rohbytes als HEX und ASCII,
- Funktion zum Leeren des Raw-Ringpuffers und Bytezaehlers,
- direkten Link zur UART-Einstellungsseite.

Die Livewerte werden ueber `/uart/status` als JSON einmal pro Sekunde aktualisiert. Die Seite prueft zusaetzlich, ob wirklich `application/json` geliefert wird und zeigt den Status der API sichtbar an. Die initialen Byte-/Paketwerte werden bereits serverseitig mit dem aktuellen Stand befuellt, sodass die Anzeige auch vor dem ersten JavaScript-Refresh konsistent ist. Die Dekodierung laeuft unabhaengig davon permanent im Hintergrund.

UART Einstellungen `/uart/settings``

Die separate Einstellungsseite enthaelt:

- Modus Aus, Raw / Sniffer (Dauerbetrieb) oder Protokoll-Decoder (Dauerbetrieb),
- Auswahl Hardware-UART1 oder UART2,
- RX- und optionalen TX-GPIO,
- Baudrate und Format 8N1, 8E1, 8O1, 8N2,
- Alias-Textfelder fuer Feld 1 bis Feld 6,
- Min/Mitte/Max-Kalibrierung fuer Feld 1 bis Feld 5,
- einstellbare Totzone,
- Anzeige des aktuellen gespeicherten Modus und UART-Status.

Beim Speichern werden die Werte in NVS abgelegt und nur die UART-Schnittstelle neu initialisiert. Ein gespeicherter Raw- oder Decoder-Modus wird beim naechsten ESP32-Start automatisch wieder aktiviert.

Web-Routen

Route	Methode	Funktion
<code>/uart/status</code>	GET	JSON-Livestatus; wird zuerst registriert, damit keine kuerzere UART-Route die API abfaengt

/uart/settings	GET	Explizite UART-Konfiguration, Feld-Aliase und Kalibrierung
/uart/monitor	GET	Reine UART-Liveansicht / Decoder-Monitor
/uart	GET	Kompatibler Redirect auf /uart/monitor; wird als letzte UART-GET-Route registriert
/save_uart	POST	Einstellungen, Aliase und Kalibrierung in NVS speichern; UART neu initialisieren
/uart_clear	POST	Raw-Ringpuffer und Bytezaehler leeren; Decoderbetrieb bleibt aktiv

Ursache des beobachteten Fehlers und Korrektur

Die Screenshots des Hardwaretests zeigen zwei Symptome, die dieselbe Ursache haben:

8. Aufruf von /uart/settings, aber Anzeige der Monitorseite.
9. Raw-Daten sind sichtbar, waehrend Byte-/Paketaehler auf 0 bleiben und keine dekodierten Felder erscheinen.

Die Raw-Daten wurden serverseitig direkt aus dem Ringpuffer in die HTML-Seite geschrieben. Die Zaehler und Decoderfelder sollten dagegen anschliessend ueber JavaScript von /uart/status aktualisiert werden. Da die kurze Route /uart die Unterroute /uart/status abfing, erhielt fetch() eine HTML-Seite statt JSON. Der bisherige JavaScript-Fehlerpfad unterdrueckte die Ausnahme; deshalb blieb die Anzeige bei den statisch vorgegebenen Nullwerten.

Die Korrektur in v0.9 besteht aus vier Massnahmen:

- spezifische /uart/*-Routen vor /uart registrieren,
- eigene Live-Route /uart/monitor verwenden,
- /uart nur als spaeten Redirect beibehalten,
- Live-JavaScript um sichtbare API-/Content-Type-Diagnose erweitern.

Da in den Raw-Daten bereits wiederholt FF FB 10 15 sichtbar ist, sollte der Decoder nach funktionierender Status-API Paketaehler und Felddaten liefern. Die implementierte XOR-Regel passt zu den im Screenshot sichtbaren Beispielpaketen; der weitere Hardwaretest bleibt dennoch massgeblich.

MQTT-Erweiterung

Der normale Heartbeat veroeffentlicht zusaetzlich den UART-Modus und den empfangenen Bytezaehler. Unter <Basis>/uart/ werden ausgegeben:

- mode
- status
- bytes
- packets
- valid_packets
- invalid_packets
- field1/raw bis field6/raw, sobald ein Hauptpaket dekodiert wurde
- field1/norm bis field5/norm
- field1/alias bis field6/alias retained

Der Raw-HEX-Datenstrom wird bewusst nicht vollstaendig ueber MQTT gespiegelt, um Broker und Funkverbindung nicht unnoetig zu belasten.

Persistenz / NVS

UART-Einstellungen werden im separaten Namespace `uartconf` gespeichert. Gespeichert werden Betriebsart, Hardware-UART, RX/TX, Baudrate, Frame, Totzone, sechs Aliase und die Kalibrierwerte fuer Feld 1 bis Feld 5. Fehlende Keys werden ohne NVS-Fehlermeldungen mit sicheren Defaults belegt.

Elektrische Voraussetzungen

Der ESP32 arbeitet mit 3,3-V-Logik und seine GPIOs sind nicht 5-V-tolerant. Vor dem Anschluss gilt:

- gemeinsame Masse zwischen ESP32 und Zielgeraet herstellen,
- zuerst nur RX anschliessen und passiv sniffen,
- High-Pegel der UART-Leitung mit Multimeter/Oszilloskop pruefen,
- bei 5-V-UART Pegelwandler oder geeigneten Spannungsteiler verwenden,
- TX erst nach geklaerter Richtung und Bus-Topologie verbinden.

Projektstruktur v0.9

```
Uart_ESP32/
.gitignore
platformio.ini
README.md
include/
  arduino_secrets.example.h
  arduino_secrets.h      # lokal, nicht in Git / nicht im Auslieferungs-ZIP
src/
  ConfigDefaults.h
  BatteryMonitor.cpp/h
  DeepSleepManager.cpp/h
  MqttManager.cpp/h
  OtaManager.cpp/h
  UartManager.cpp/h      # neu v0.7
  main.cpp
docs/
  2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.9.*
  2026-09-17 - Uart_Esp32 - Commit-Nachricht - UART-Seitentrennung-und-Dauerbetrieb - v0.9.txt
```

Hinweis: Der Projektname bleibt Uart_Esp32. Der Root-Ordner Uart_ESP32 wird fuer diese Auslieferung unveraendert aus dem vom Nutzer bereitgestellten Paket uebernommen, entsprechend der Regel zur Beibehaltung des Codepaket-Root-Ordners.

Geaenderte / neue Dateien in v0.9

- README.md
- src/ConfigDefaults.h
- src/main.cpp
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.9.md (neu)
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.9.docx (neu)
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.9.pdf (neu)
- docs/2026-09-17 - Uart_Esp32 - Commit-Nachricht - UART-Routing-und-Live-Status - v0.9.txt (neu)

Pruefung und offene Verifikation

Die Aenderungen wurden statisch auf konsistente Einbindung, NVS-Key-Laengen, Parsergrenzen, Pinvalidierung und die Dokumentationsstruktur geprueft. Eine echte PlatformIO-Kompilierung mit dem ESP32-Arduino-Framework ist in der Erstellungsumgebung nicht verfuegbar. Verbindlicher lokaler Test:

```
pio run -e uart_esp32_usb -t clean
pio run -e uart_esp32_usb
pio run -e uart_esp32_usb -t upload
```

Danach sollte auf /uart/settings zunaechst Raw / Sniffer (Dauerbetrieb) eingestellt werden. Auf /uart/monitor muss anschliessend Status-API: OK erscheinen und die Bytezahl kontinuierlich steigen. Fuer die erste Hardwaremessung TX auf -1 belassen und nur RX sowie gemeinsame Masse verbinden. Erst nach bestaetigter 3,3-V-Pegellage den Protokoll-Decoder (Dauerbetrieb) mit 115200 8N1 aktivieren. Die Monitorseite darf geschlossen werden; der Decoder muss weiterhin laufen.

Auslieferungsform

Bereitgestellt werden ein vollstaendiges ZIP mit unveraendertem Root-Ordner Uart_ESP32, ein ZIP mit nur geaenderten/neuen Dateien in derselben Struktur, eine Patch-Datei, eine englische Commit-Nachricht mit Datum und vollstaendiger Dateiliste sowie die Projektdokumentation als PDF und bearbeitbare DOCX mit identischem Stand v0.9.